



**RV
UNIVERSITY**

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

**New-Age Global University
for Liberal Education**

School of Computer Science and Engineering

Where Ideas Ignite Minds

Laboratory Record Book CS1006 - Data Structures

Name:

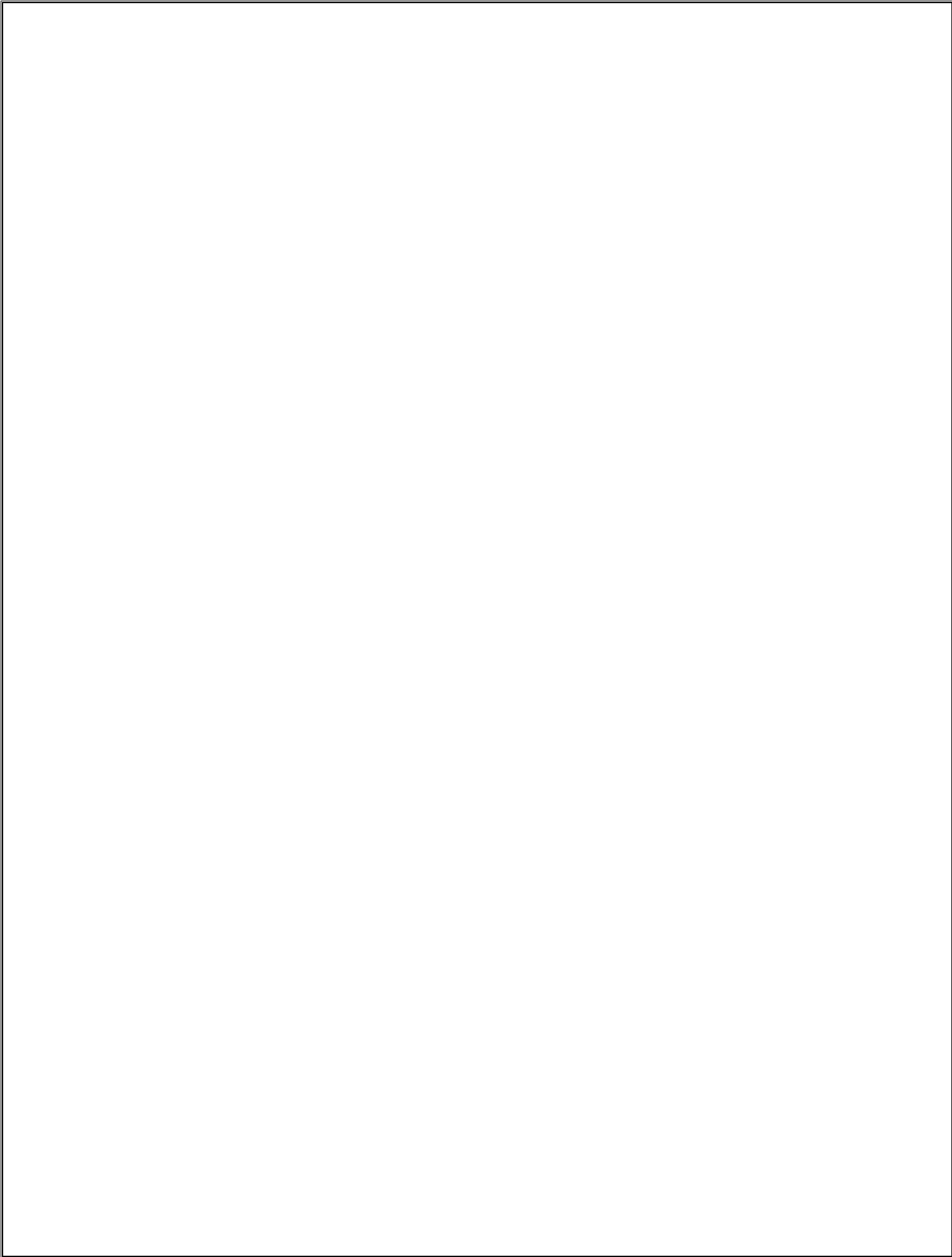
USN:

Section:

Bachelor of Technology (B.Tech) (Hons.)

Academic Year 2025-26

RV UNIVERSITY



Rashtreeya Sikshana Samithi Trust
RV UNIVERSITY
School of Computer Science and Engineering
Bengaluru – 560059



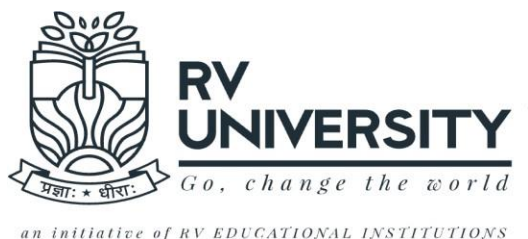
DATA STRUCTURES

COURSE CODE: CS1006

II SEMESTER B.Tech (Hons.)

LABORATORY RECORD 2025-2026

RV UNIVERSITY
School of Computer Science and Engineering
Bengaluru – 560 059



LABORATORY CERTIFICATE

This is to certify that Mr./Ms. _____ has satisfactorily completed the course of experiments in Practical **Data Structures (CS1006)** prescribed by the **School of Computer Science and Engineering** during the year **2025-26**.

Name of the Candidate: _____

USN: _____ **Semester:** _____

Marks	
Maximum	Obtained
10	

Signature of Faculty In-charge

Program Director

Date:

Vision and Mission of the School of Computer Science and Engineering

Vision

To be a pioneering school of Computer Science and Engineering committed to fostering liberal education and empowering the next generation of technologists to make a positive global socio-economic impact.

Mission

- To be a pioneer in computer science education and benchmark ourselves with the world's top computer science and engineering institutions.
- To provide state-of-the-art facilities that enable exemplary pedagogy, advanced research, innovation and entrepreneurship in emerging technologies of computer science.
- To promote a culture of cooperation and inclusiveness among students and faculty from diverse communities enabling them to take part in interdisciplinary and multidisciplinary research, contributing to institution-building.
- To foster excellence through national and international academic, industry collaborations, bringing in diverse perspectives to drive innovation.
- To nurture a talented pool of ethical, self-driven and empathetic problem solvers to achieve sustainable development goals.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- PEO1:** Graduates will be able to demonstrate excellence in the field of Computer Science and Engineering through advanced research and entrepreneurship.
- PEO2:** Graduates will be able to achieve excellence in innovation through national and international academic and industry collaborations.
- PEO3:** Graduates will be able to actively engage in cutting-edge interdisciplinary and multidisciplinary research.
- PEO4:** Graduates will be able to function effectively as talented pool of ethical, sustainable, self-driven and life-long empathetic problem solvers.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

- PO1: Engineering Knowledge.** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO2: Problem Analysis.** Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO3: Design/Development of Solutions.** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO4: Conduct Investigations of Complex Problems.** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO5: Modern Tool Usage.** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
- PO6: The Engineer and Society.** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues, and the consequent responsibilities relevant to the professional engineering practice.
- PO7: Environment and Sustainability.** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO8: Individual and Teamwork.** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO9: Ethics.** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

- PO10: Communication.** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO11: Project Management and Finance.** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- PO12: Life-Long Learning.** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Outcomes: After completing the course, the students will be able to:	
CO1	Understand and explore the concepts of linear and non-linear data structures to develop solutions for diverse problems
CO2	Apply the concepts of hashing, sorting and searching techniques to get an optimized solution
CO3	Evaluate and apply appropriate data structure(s) for real-world problems
CO4	Demonstrate Good Coding Practices involving lifelong learning

LIST OF PROGRAMS

S.No.	Name of the Program	Week
1 (a)	Write a program to implement recursion for Decimal to Binary Conversion	Week 2
(b)	Write a program to implement recursion for printing pyramid pattern using recursion	
(c)	Write a program to solve Tower of Hanoi Problem with 'n' disks using recursion (T)	
2 (a)	Write a program to implement arrays and their operations.	Week 3
(b)	Write a program using array to find two elements whose sum is closest to zero. (T)	
3 (a)	Write a program to implement Stack and its operations.	Week 4
(b)	Write a Program to Convert Infix to Postfix Conversion using Stack	
(c)	Write a Program to implement Postfix Evaluation using Stack (T)	
4 (a)	Write a program to implement Queue and its operations.	Week 5
(b)	Write a program to implement First Come First Serve CPU Scheduling Algorithm (T)	
5	Write a program to implement Singly Linked List and its operations.	Week 6
6	Write a program to implement Doubly Linked List and its operations.	Week 7
7	Write a program to implement Circular Singly Linked List and its operations.	Week 8
8 (a)	Write a program to implement Circular Doubly Linked List and its operations.	Week 9
(b)	Write a program to implement the application of Linked List – Polynomial Addition (T)	
9 (a)	Write a program to implement binary search tree creation and traversals.	Week 10
(b)	Write a program to implement binary search tree – Searching and deletion (T)	
10 (a)	Write a program to implement Linear Search	Week 11
(b)	Write a program to implement Binary Search (T)	

11 (a)	Write a program to implement Insertion sort	Week 12
(b)	Write a program to implement Bubble Sort (T)	
12 (a)	Write a program to implement hash table using Linear Probing	Week 13
(b)	Write a program to implement hash table using Quadratic Probing (T)	
13	Write a program to implement AVL Tree and its Rotations (T)	Week 14
14	Write a program to implement B-Trees – Creation, Searching and Deletion (T)	

INDEX

S. No.	Name of the Program	Marks		Signature Faculty In-charge with date
		Record (6 Marks)	Viva-voce (4 Marks)	
1 (a)	Write a program to implement recursion for Decimal to Binary Conversion			
(b)	Write a program to implement recursion for printing pyramid pattern using recursion			
(c)	Write a program to solve Tower of Hanoi Problem with 'n' disks using recursion (T)			
2 (a)	Write a program to implement arrays and their operations.			
(b)	Write a program using array to find two elements whose sum is closest to zero. (T)			
3 (a)	Write a program to implement Stack and its operations.			
(b)	Write a Program to Convert Infix to Postfix Conversion using Stack			
(c)	Write a Program to implement Postfix Evaluation using Stack (T)			
4 (a)	Write a program to implement Queue and its operations.			
(b)	Write a program to implement First Come First Serve CPU Scheduling Algorithm (T)			
5	Write a program to implement Singly Linked List and its operations.			
6	Write a program to implement Doubly Linked List and its operations.			
7	Write a program to implement Circular Singly Linked List and its operations.			
8 (a)	Write a program to implement Circular Doubly Linked List and its operations.			
(b)	Write a program to implement the application of Linked List – Polynomial Addition (T)			
9 (a)	Write a program to implement binary search tree creation and traversals.			
(b)	Write a program to implement binary search tree – Searching and deletion (T)			
10 (a)	Write a program to implement Linear Search			
(b)	Write a program to implement Binary Search (T)			
11 (a)	Write a program to implement Insertion sort			

(b)	Write a program to implement Bubble Sort (T)			
12 (a)	Write a program to implement hash table using Linear Probing			
(b)	Write a program to implement hash table using Quadratic Probing (T)			
13	Write a program to implement AVL Tree and its Rotations (T)			
14	Write a program to implement B-Trees – Creation, Searching and Deletion (T)			
TOTAL MARKS OBTAINED				

Rubrics

Lab Write-Up and Execution Rubrics (Max: 6 Marks)					
S.No.	Criteria	Measuring Methods	Excellent	Good	Poor
1	Understanding of the problem statement (2 Marks)	Lab Note Book	Student exhibits thorough understanding of the problem statement. (2 Marks)	Student has sufficient understanding of the problem statement. (1 Mark)	Student does not have a clear understanding of the problem statement. (0 Marks)
2	Execution and passing the test cases (2 Marks)	Lab Note Book	Student demonstrates the execution of the program with optimized code. Appropriate validations passing all the test cases. (2 Marks)	Student demonstrates the execution of the program without optimized code and passing only a few of the test cases. (1 Mark)	Student implemented but has not executed the program. None of the test cases passed. (0 Mark)
3	Results and Documentation (2 Marks)	Lab Note Book	Documentation with appropriate comments and output is covered in manual. (2 Marks)	Documentation with only few comments and few outputs are covered in manual. (1 Mark)	Documentation with no comments and no output cases is covered in manual. (0 Mark)

Viva-Voce Rubrics (Max: 4 Marks)					
1	Conceptual Understanding (2 Marks)	Viva Voce	Explains thoroughly the solution along with related concepts. (2 Marks)	Adequately explains the solution and the related concepts (1Mark)	Unable to explain the solution (0 Mark)
2	Use of techniques (1 Mark)	Viva Voce	Insightful explanation of the appropriate data structure for the given problem to derive a solution (1 Mark)	Sufficient explanation of the appropriate data structure for the given problem to derive a solution (0.5 Mark)	Unable to explain the data structure for the given problem (0 Mark)
3	Communication of Concepts (1 Mark)	Viva Voce	Communicates the concept used in problem solving well. (1 Mark)	Sufficiently communicates the concepts used in problem solving. (0.5 Mark)	Unable to communicate the concepts used in problem. (0 Mark)

Ex NO: 1(a)	IMPLEMENTATION OF DECIMAL TO BINARY CONVERSION USING RECURSION
DATE :	

Aim

To write a 'C' program to implement Decimal to Binary Conversion using recursion.

Basic Concept:

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function. Using a recursive algorithm, certain problems can be solved quite easily.

Properties of Recursion:

- Performing the same operations multiple times with different inputs.
- In every step, smaller inputs are taken to make the problem smaller.
- Base condition is needed to stop the recursion otherwise infinite loop will occur.

Algorithm

1. Start
2. Base Case: If the number is 0, return 0.
3. Recursive Case: Otherwise, return the remainder of the number divided by 2 plus 10 times the result of the function called with the number divided by 2.
4. Stop

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement decimal to binary conversion using recursion has been executed and the output is verified successfully.

Ex NO: 1(b)	IMPLEMENTATION OF PYRAMID PATTERN USING RECURSION
DATE :	

Aim

To write a 'C' program to print pyramid pattern using recursion.

Algorithm

1. Start
2. Set the height of the pyramid
3. Adjust space using recursion function
4. Adjust Hash(#) character using recursion function
5. Call both functions altogether to print the Pyramid pattern
6. Stop

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to print pyramid pattern using recursion has been executed and the output is verified successfully.

Ex NO: 1(c)	TOWER OF HANOI USING RECURSION
DATE :	

Aim

To write a 'C' program to implement Tower of Hanoi with 'n' disks using recursion.

Concept:

Tower of Hanoi is a mathematical puzzle where we have three rods (**A**, **B**, and **C**) and **N** disks. Initially, all the disks are stacked in decreasing value of diameter i.e., the smallest disk is placed on the top and they are on rod **A**. The objective of the puzzle is to move the entire stack to another rod (**C**), obeying the following simple rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
- No disk may be placed on top of a smaller disk.

Algorithm

1. Start
2. Create a function **towerOfHanoi** where pass the **N** (current number of disk), **from_rod**, **to_rod**, **aux_rod**.
3. Make a function call for $N - 1$ th disk.
4. Then print the current the disk along with **from_rod** and **to_rod**.
5. Again make a function call for $N - 1$ th disk.
6. Stop

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Tower of Hanoi problem with 'n' disks using recursion has been executed and the output is verified successfully.

Ex NO: 2(a)	IMPLEMENTATION OF ARRAY AND ITS OPERATIONS
DATE :	

Aim

To write a 'C' program to implement array and its operations such as insert, display, search update and delete.

Basic Operations in Arrays

The basic operations in the Arrays are insertion, deletion, searching, display, traverse, and update. These operations are usually performed to either modify the data in the array or to report the status of the array.

Following are the basic operations supported by an array.

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Display** – Displays the contents of the array.

Algorithm

Insertion

1. Start
2. Create an Array of a desired datatype and size.
3. Initialize a variable 'i' as 0.
4. Enter the element at ith index of the array.
5. Increment i by 1.
6. Repeat Steps 4 & 5 until the end of the array.
7. Stop

Deletion

1. Start
2. Set J = K
3. Repeat steps 4 and 5 while J < N
4. Set LA[J] = LA[J + 1]

5. Set $J = J+1$
6. Set $N = N-1$
7. Stop

Search

1. Start
2. Set $J = 0$
3. Repeat steps 4 and 5 while $J < N$
4. IF $LA[J]$ is equal ITEM THEN GOTO STEP 6
5. Set $J = J + 1$
6. PRINT J, ITEM
7. Stop

Traversal

- 1 Start
2. Initialize an Array of certain size and datatype.
3. Initialize another variable 'i' with 0.
4. Print the ith value in the array and increment i.
5. Repeat Step 4 until the end of the array is reached.
6. End

Update

1. Start
2. Set $LA[K-1] = \text{ITEM}$
3. Stop

Display

1. Start
2. Print all the elements in the Array
3. Stop

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement array and its operations such as create,insert,display,search delete has been executed and the output is verified successfully.

Ex NO: 2(b)	IMPLEMENTATION OF ARRAY TO FIND TWO ELEMENTS WHOSE SUM IS CLOSEST TO ZERO
DATE :	

Aim

To write a 'C' program to implement array to find two elements such that their sum is closest to zero.
update

Algorithm

Step 1: Start

Step 2: Initialize the array with some numbers.

Step 3: Add the first two elements of array and let it be the minimum sum.

Step 4: Keeping this as default, try all the combinations in the array. Use for loops for the
execution.

Step 5: Find the minimum and print the output.

Step 6: Stop

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement array to find two elements such that the sum is closest to zero has been executed and the output is verified successfully.

Ex NO: 3(a)	IMPLEMENTATION OF STACK AND ITS OPERATIONS
DATE :	

Aim

To write a 'C' program to implement Stack and its operations such as Push,Pop,Peak,display, delete,Stack full and Stack empty.

Algorithm for Push Operation:

1. Before pushing the element to the stack, we check if the stack is **full** .
2. If the stack is full (**top == capacity-1**) , then **Stack Overflows** and we cannot insert the element to the stack.
3. Otherwise, we increment the value of top by 1 (**top = top + 1**) and the new value is inserted at **top position** .
4. The elements can be pushed into the stack till we reach the **capacity** of the stack.

Algorithm for Pop Operation:

1. Before popping the element from the stack, we check if the stack is **empty** .
2. If the stack is empty (**top == -1**), then **Stack Underflows** and we cannot remove any element from the stack.
3. Otherwise, we store the value at top, decrement the value of top by 1 (**top = top - 1**) and return the stored top value.

Algorithm for Top/Peak Operation:

1. Before returning the top element from the stack, we check if the stack is empty.
2. If the stack is empty (**top == -1**), we simply print "Stack is empty".
3. Otherwise, we return the element stored at **index = top** .

Algorithm for isEmpty Operation:

1. Check for the value of **top** in stack.
2. If (**top == -1**) , then the stack is **empty** so return **true** .
3. Otherwise, the stack is not empty so return **false** .

Algorithm for isFull Operation:

1. Check for the value of **top** in stack.
2. If (**top == capacity-1**), then the stack is **full** so return **true**.
3. Otherwise, the stack is not full so return **false**.

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Stack and its operations such as Push,Pop,Peak,display, delete,Stack full and Stack empty has been executed and the output is verified successfully.

Ex NO: 3(b)	IMPLEMENTATION OF INFIX TO POSTFIX CONVERSION
DATE :	

Aim

To write a 'C' program to implement Infix to Postfix Conversion of an expression using Stack.

Algorithm

Step 1: Start the program

Step 2: Let Q be any infix expression and we have to convert it to postfix expression P.

Step 3: Push left parenthesis onto STACK and add right parenthesis at the end of Q.

Step 4: Scan Q from left to right and repeat step 3 to 6 for each element of Q until the STACK is empty.

Step 5: If an operand is encountered add it to P.

Step 6: If a left parenthesis is encountered push it onto the STACK.

Step 7: If an operator is encountered, then

- a. Repeatedly pop from STACK and add to P each operator which has same precedence as or higher precedence than the operator encountered.
- b. Push the encountered operator onto the STACK.

Step 8: If a right parenthesis is encountered, then

- a. Repeatedly pop from the STACK and add to P each operator until a left parenthesis is encountered.
- b. Remove the left parenthesis; do not add it to P.

Step 9: Stop the execution

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Infix to Postfix Conversion of an expression using Stack has been executed and the output is verified successfully.

Ex NO: 3(c)	IMPLEMENTATION OF POSTFIX EVALUATION
DATE :	

Aim

To write a 'C' program to implement Postfix Evaluation of an expression using Stack.

Algorithm

Step 1: Start the program

Step 2: Create an empty stack.

Step 3: Scan the postfix expression from left to right.

Step 4: If an element is an operand, push it into the stack.

Step 5: If an element is an operator, pop twice to get operands A and B.

Step 6: Calculate the result of B O A (where O is the operator) and push it back to the stack.

Step 7: Repeat until the end of the expression

Step 8 : Stop the execution

.

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Postfix Evaluation of an expression using Stack has been executed and the output is verified successfully.

Ex NO: 4(a)	IMPLEMENTATION OF QUEUE AND ITS OPERATIONS
DATE :	

Aim

To write a 'C' program to implement Queue and its operations such as – Enqueue, Dequeue, display, search, queue_full and queue_empty.

Algorithm

Enqueue Operation:

Enqueue() operation in Queue **adds (or stores) an element to the end of the queue.**

The following steps should be taken to enqueue (insert) data into a queue:

Step 1: Check if the queue is full.

Step 2: If the queue is full, return overflow error and exit.

Step 3: If the queue is not full, increment the rear pointer to point to the next empty space.

Step 4: Add the data element to the queue location, where the rear is pointing.

Step 5: return success.

Dequeue Operation:

Removes (or access) the first element from the queue.

The following steps are taken to perform the dequeue operation:

Step 1: Check if the queue is empty.

Step 2: If the queue is empty, return the underflow error and exit.

Step 3: If the queue is not empty, access the data where the front is pointing.

Step 4: Increment the front pointer to point to the next available data element.

Step 5: The Return success.

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Queue and its operations such as – Enqueue, Dequeue, display, search, queue_full and queue_empty has been executed and the output is verified successfully.

Ex NO: 4(b)	IMPLEMENTATION OF FCFS SCHEDULING ALGORITHM
DATE :	

Aim

To write a 'C' program to implement First come First Serve Scheduling Algorithm as an application of Queue data structure.

Concept:

First Come First Serve CPU Scheduling Algorithm shortly known as FCFS is the first algorithm of CPU Process Scheduling Algorithm. In First Come First Serve Algorithm what we do is to allow the process to execute in linear manner. This means that whichever process enters process enters the ready queue first is executed first. This shows that First Come First Serve Algorithm follows First In First Out (FIFO) principle.

Terminologies Used in CPU Scheduling

- Arrival Time: The time at which the process arrives in the ready queue.
- Completion Time: The time at which the process completes its execution.
- Turn Around Time: Time Difference between completion time and arrival time. Turn Around Time = (Completion Time – Arrival Time)
- Waiting Time (W. T): Time Difference between turnaround time and burst time I.e.
- *Waiting Time = (Turn Around Time – Burst Time).*

Algorithm

1. Start
2. Input the processes along with their burst time (bt).
3. Find waiting time (wt) for all processes.
4. As first process that comes need not to wait so waiting time for process 1 will be 0 i.e. $wt[0] = 0$.
5. Find **waiting time** for all other processes i.e. for process $i \rightarrow wt[i] = bt[i-1] + wt[i-1]$.
6. Find **turnaround time** = waiting_time + burst_time for all processes.
7. Find **average waiting time** = total_waiting_time / no_of_processes.
8. Similarly, find **average turnaround time** = total_turn_around_time / no_of_processes.
9. Stop the process

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement FCFS scheduling algorithm has been executed and the output is verified successfully.

Ex NO: 5	IMPLEMENTATION OF SINGLY LINKED LIST AND ITS OPERATIONS
DATE :	

Aim

To write a 'C' program to implement Singly Linked List and its operations such as create,insert,display,search and delete.

Algorithm

Step 1: Start

Step 1: Include all the header files which are used in the program.

Step 2: Declare all the user defined functions.

Step 3: Define a Node structure with two members data and next

Step 4: Define a Node pointer 'head' and set it to NULL.

Step 4: Implement the main method by displaying operations menu and make suitable function calls in the main method to perform user selected operation.

Step 6: Stop

Insertion

In a single linked list, the insertion operation can be performed in three ways. They are as follows...

- Inserting At Beginning of the list
- Inserting At End of the list
- Inserting At Specific location in the list
- Inserting At Beginning of the list

Inserting At Beginning of the list

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL)

Step 3: If it is Empty then, set newNode→next = NULL and head = newNode.

Step 4: If it is Not Empty then, set newNode→next =head and head=newNode

Inserting At End of the list

Step 1: Create a newNode with given value and newNode → next as NULL.

Step 2: Check whether list is Empty (head == NULL).

Step 3: If it is Empty then, set head = newNode.

Step 4: If it is Not Empty then, define a node pointer temp and initialize with head.

Step 5: Keep moving the temp to its next node until it reaches to the last node in the list (until temp → next is equal to NULL).

Step 6: Set temp → next = newNode.

Inserting At Specific location in the list (After a Node)

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL)

Step 3: If it is Empty then, set newNode → next = NULL and head = newNode.

Step 4: If it is Not Empty then, define a node pointer temp and initialize with head.

Step 5: Keep moving the temp to its next node until it reaches to the node after which we want to insert the newNode (until temp1 → data is equal to location, here location is the node value after which we want to insert the newNode).

Step 6: Every time check whether temp is reached to last node or not. If it is reached to last node then display 'Given node is not found in the list!!! Insertion not possible!!!' and terminate the function. Otherwise move the temp to next node.

Step 7: Finally, Set 'newNode → next = temp → next' and 'temp → next = newNode'

Deletion

In a single linked list, the deletion operation can be performed in three ways. They are as follows...

- Deleting from Beginning of the list
- Deleting from End of the list
- Deleting a Specific Node
- Deleting from Beginning of the list

Deleting from Beginning of the list

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define a Node pointer 'temp' and initialize with head.

Step 4: Check whether list is having only one node (temp → next == NULL)

Step 5: If it is TRUE then set head = NULL and delete temp (Setting Empty list conditions)

Step 6: If it is FALSE then set head = temp → next, and delete temp.

Deleting from End of the list

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.

Step 4: Check whether list has only one Node (temp1 → next == NULL)

Step 5: If it is TRUE. Then, set head = NULL and delete temp1. And terminate the function.
(Setting Empty list condition)

Step 6: If it is FALSE. Then, set 'temp2 = temp1 ' and move temp1 to its next node. Repeat the same until it reaches to the last node in the list (until temp1 → next == NULL)

Step 7: Finally, Set temp2 → next = NULL and delete temp1.

Deleting a Specific Node from the list

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.

Step 4: Keep moving the temp1 until it reaches to the exact node to be deleted or to the last node. And every time set 'temp2 = temp1' before moving the 'temp1' to its next node.

Step 5: If it is reached to the last node then display 'Given node not found in the list! Deletion not possible!!!'. And terminate the function.

Step 6: If it is reached to the exact node which we want to delete, then check whether list is having only one node or not

Step 7: If list has only one node and that is the node to be deleted, then set head = NULL and delete temp1 (free(temp1)).

Step 8: If list contains multiple nodes, then check whether temp1 is the first node in the list (temp1 == head).

Step 9: If temp1 is the first node then move the head to the next node (head = head → next) and delete temp1.

Step 10: If temp1 is not first node then check whether it is last node in the list (temp1 → next == NULL).

Step 11: If temp1 is last node then set temp2 → next = NULL and delete temp1 (free(temp1)).

Step 12: If temp1 is not first node and not last node then set temp2 → next = temp1 → next and delete temp1 (free(temp1)).

Displaying a Single Linked List

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty!!!' and terminate the function.

Step 3: If it is Not Empty then, define a Node pointer 'temp' and initialize with head.

Step 4: Keep displaying temp → data with an arrow (--->) until temp reaches to the last node

Step 5: Finally display temp → data with arrow pointing to NULL (temp → data ---> NULL).

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Linked List and its operations such as create,insert,display, search and delete has been executed and the output is verified successfully.

Ex NO: 6	IMPLEMENTATION OF DOUBLY LINKED LIST AND ITS OPERATIONS
DATE :	

Aim

To write a 'C' program to implement Doubly Linked List and its operations.

Algorithm**Insertion at the Beginning of the Doubly Linked List**

1. Create a newNode with **data** field assigned the given value.
2. If the list is empty:
 - a. Set newNode->next to NULL.
 - b. Set newNode->prev to NULL.
 - c. Update the head pointer to point to newNode.
3. If the list is not empty:
 - a. Set newNode->next to head.
 - b. Set newNode->prev to NULL.
 - c. Set head->prev to newNode.
 - d. Update the head pointer to point to newNode.

Insertion at the End of the Doubly Linked List

1. Create a newNode with **data** field assigned the given value.
2. If the list is empty:
 - a. Set newNode->next to NULL.
 - b. Set newNode->prev to NULL.
 - c. Update the head pointer to point to newNode.
3. If the list is not empty:
 - a. Traverse the list to find the last node (where last->next == NULL).
 - b. Set last->next to newNode.
 - c. Set newNode->prev to last.
 - d. Set newNode->next to NULL.

Insertion in the Middle of the Doubly Linked List

1. Create a newNode.
2. Find the size of the list to ensure the position is within valid bounds.
3. If the position is 0 (or 1 depending on your indexing):

- a. Use the insertion at the beginning approach.
4. If the position is equal to the size of the list:
 - a. Use the insertion at the end approach.
5. If the position is valid and not at the boundaries:
 - a. Traverse the list to find the node immediately before the desired insertion point (prevNode).
 - b. Set newNode->next to prevNode->next.
 - c. Set newNode->prev to prevNode.
 - d. If prevNode->next is not NULL, set prevNode->next->prev to newNode.
 - e. Set prevNode->next to newNode.

Deletion at the Beginning of the Doubly Linked List

1. **Check if the List is Empty:** If true, there's nothing to delete.
2. **Check if the List Contains Only One Node:**
 - a. Set head to NULL.
 - b. Free the memory of the node.
3. **If the List Contains More Than One Node:**
 - a. Update head to head->next.
 - b. Set the prev of the new head to NULL.
 - c. Free the memory of the old head.

Deletion at the End of the Doubly Linked List

1. Check if the List is Empty: If true, there's nothing to delete.
2. If the List is Not Empty:
 - a. Traverse to the last node using a loop.
 - b. Check if the last node is the only node (head->next == NULL):
 - i. Update head to NULL.
 - c. If more than one node:
 - i. Set the next of the second last node (last->prev) to NULL.
 - d. Free the memory of the last node.

Deletion at a Specific Position in Doubly Linked List

Check Position Validity:

1. If position is 0 (or 1 depending on indexing), use deletion at the beginning.
2. If position is the size of the list, use deletion at the end.
3. Otherwise, proceed with the middle deletion.

Traverse to the Specified Position:

1. Use a loop to reach the node (delNode) at the desired position.
2. If delNode is the first node, adjust head.
3. If not, adjust delNode->prev->next and delNode->next->prev if delNode->next is not NULL.

Free the Memory:

1. Release the node to be deleted.

Traversal in a Doubly Linked List in C**Forward Traversal in Doubly Linked List**

1. Create a temporary pointer ptr and copy the head pointer into it.
2. Traverse through the list using a while loop.
3. Keep moving the value of temp pointer variable ptr to ptr->next until the last node whose next part contains null is found.
4. During each iteration of the loop, perform the desired operation.

Reverse Traversal in Doubly Linked List

1. Create a temporary pointer ptr and copy the tail pointer into it.
2. Traverse through the list using a while loop.
3. Keep moving the value of temp pointer variable ptr to ptr->prev until the first node whose prev part contains null is found.
4. During each iteration of the loop, perform the desired operation .

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Doubly Linked list and its operations has been executed and the output is verified successfully.

Ex NO: 7	IMPLEMENTATION OF SINGLY CIRCULAR LINKED LIST AND ITS OPERATIONS
DATE :	

Aim

To write a 'C' program to implement Singly Circular Linked List and its operations.

Algorithm

Implementation of InsertAtBegin

To insert a node at the beginning of a circular linked list:

1. Create a new node with the given data.
2. If the list is empty, set the new node to itself and make it the head node.
3. If the list is not empty, traverse to the last node and update its next pointer to point to the new node. Then update the head pointer to the new node.'

Implementation of InsertAtEnd

To insert a node at the end of a circular linked list:

1. Create a new node with the given data.
2. If the list is empty, set the new node to itself and make it the head node.
3. If the list is not empty, traverse to the last node.
4. Set the last node's next pointer to the new node.
5. Set the new node's next pointer to the head to maintain circularity.

Implementation of InsertAtSpecificPosition

To insert a node at a specific position in a circular linked list:

1. Create a new node with the given data.
2. If the list is empty and position is 0, set the new node to itself and make it the head node.
3. Traverse to the node just before the desired position.
4. Insert the new node by adjusting pointers.

Implementation of DeleteFromBeginning

To delete a node from the beginning of a circular linked list:

1. If the list is empty, return.
2. If the list has only one node, delete it and set head to NULL.
3. Traverse to the last node and update its next pointer.
4. Free the memory allocated to the node to be deleted.

Implementation of DeleteFromEnd

To delete a node from the end of a circular linked list:

1. If the list is empty, return.
2. If the list has only one node, delete it and set head to NULL.
3. Traverse to the second last node and update its next pointer to NULL.
4. Free the memory allocated to the last node.

Implementation of DeleteFromSpecificPosition

To delete a node from a specific position in a circular linked list:

1. If the list is empty, return.

2. Traverse to the node just before the desired position.
3. Adjust pointers to skip the node to be deleted.
4. Free the memory allocated to the node to be deleted.

Implementation of Traversal

1. Check if list is empty then if it is return.
2. Initialize the temporary pointer to the head.
3. Loop through the list starting from head and print the each nodes data.
4. Continue until the pointer reaches back to head.

4. Search

To search for a node with a given key in a circular linked list:

1. Start from the head node.
2. Traverse each node and compare its data with the given key until the key is found or you reach the head again.

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Singly Circular Linked List and its operations has been executed and the output is verified successfully.

Ex NO: 8(a)	IMPLEMENTATION OF CIRCULAR DOUBLY LINKED LIST AND ITS OPERATIONS
DATE :	

Aim

To write a 'C' program to implement Circular Doubly Linked List and its operations.

Algorithm**Insertion before a given node in Doubly Circular Linked List – O(n) Time and O(1) Space:**

To insert a new node before a specific node in doubly circular linked list,

1. Allocate memory for the new node.
2. Traverse the list to **locate** the givenNode.
3. Insert the New Node:
 - a. Set **newNode->next** to givenNode.
 - b. Set **newNode->prev** to **givenNode->prev**.
 - c. Update **givenNode->prev->next** to newNode.
 - d. Update **givenNode->prev** to newNode.
4. Update Head (if givenNode is the head node), set **head** to newNode.

Insertion at a specific position in Doubly Circular Linked List – O(n) Time and O(1) Space:

To insert a new node at a specific position in doubly circular linked list,

1. Allocate memory for the new node.
2. Initialize a pointer **curr** pointer to the **head** node and start traversing the list we reach the node just before the desired **position**. Use a counter to keep track of the **curr** position.
3. Insert the New Node:
 - a. Set **newNode->next** to **curr->next**.
 - b. Set **newNode->prev** to **curr**.
 - c. Update **curr->next->prev** to **newNode**.
 - d. Update **current->next** to **newNode**.
4. Update **Head** (if the insertion is at position 0 and the list is empty), set **head** to newNode.

Search an Element in Doubly Circular Linked List**Algorithm:**

1. Declare a temp pointer, and initialize it to the head of the list.
2. Iterate the loop until temp reaches the start address (last node in the list, as it is in a circular fashion), and check for the *n* element, whether present or not.

3. If it is present, raise a flag, increment count, and break the loop.
4. At the last, as the last node is not visited yet check for the n element if present does step

Deletion in Doubly Circular Linked List

Algorithm:

Case 1: Empty List(start = NULL)

1. If the list is empty, simply return it.

Case 2: The List initially contains some nodes, start points at the first node of the List

1. If the list is not empty, then we define two pointers **curr** and **prev_1** and initialize the pointer **curr** points to the first node of the list, and **prev_1** = NULL.
2. Traverse the list using the curr pointer to find the node to be deleted and before moving from curr to the next node, every time set **prev_1** = curr.
3. If the node is found, check if it is the only node in the list. If yes, set start = NULL and free the node pointing by curr.
4. If the list has more than one node, check if it is the first node of the list. The condition to check this is (curr == start). If yes, then move prev_1 to the last node(prev_1 = start -> prev). After prev_1 reaches the last node, set start = start -> next and prev_1 -> next = start and start -> prev = prev_1. Free the node pointing by curr.
5. If curr is not the first node, we check if it is the last node in the list. The condition to check this is (curr -> next == start). If yes, set prev_1 -> next = start and start -> prev = prev_1. Free the node pointing by curr.
6. If the node to be deleted is neither the first node nor the last node, declare one more pointer **temp** and initialize the pointer **temp** points to the next of curr pointer (temp = curr->next). Now set, prev_1 -> next = temp and temp -> prev = prev_1. Free the node pointing by curr.

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Circular Doubly Linked list and its operations has been executed and the output is verified successfully.

Ex NO: 8 (b)	IMPLEMENTATION OF APPLICATION OF LINKED LIST – POLYNOMIAL ADDITION
DATE :	

Aim

To write a 'C' program to implement the application of Linked List – Polynomial addition.

Algorithm

Step 1. Start

Step 2: Create a linked list to store each polynomial.

Step 3: Traverse both linked lists simultaneously.

Step 4: If exponents match, add coefficients and store the result.

Step 5: If one exponent is larger, append the term to the result.

Step 6: Continue until all terms are processed.

Step 7: Return the resultant linked list.

Step 8: Stop the execution

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement the application of Linked List – Polynomial addition has been executed and the output is verified successfully.

Ex NO: 9 (a)	IMPLEMENTATION OF BINARY SEARCH TREE CREATION AND TRAVERSAL
DATE :	

Aim

To write a 'C' program to implement Binary search tree creation and its traversals.

Algorithm - Insertion

Step 1. Start.

Step 2. If the tree is empty, insert the first element as the root node of the tree. The following elements are added as the leaf nodes.

Step 3. If an element is less than the root value, it is added into the left subtree as a leaf node.

Step 4. If an element is greater than the root value, it is added into the right subtree as a leaf node.

Step 5. The final leaf nodes of the tree point to NULL values as their child nodes.

Step 6. End

INORDER

Step 1. Start

Step 2. Traverse the left subtree, recursively

Step 3. Then, traverse the root node

Step 4. Traverse the right subtree, recursively.

Step 5. End

PREORDER

Step 1. Start

Step 2. Traverse the root node first.

Step 3. Then traverse the left subtree, recursively

Step 4. Later, traverse the right subtree, recursively.

Step 5. End

POSTORDER

Step 1. Start

Step 2. Traverse the left subtree, recursively

Step 3. Traverse the right subtree, recursively.

Step 4. Then, traverse the root node

Step 5. End

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Binary Search Tree creation and its traversals has been executed and the output is verified successfully.

Ex NO: 9(b)	IMPLEMENTATION OF BINARY SEARCH TREE SEARCHING AND DELETION
DATE :	

Aim

To write a 'C' program to implement Binary Search Tree and its operations such as searching and deletion.

Algorithm - Searching

Step 1: Start.

Step 2 : Compare the element to be searched with the root node of the tree.

Step 3 : If the value of the element to be searched is equal to the value of the root node, return the root node.

Step 4 : If the value does not match, check whether the value is less than the root element or not if it is then traverse the left subtree.

Step 5 :If larger than the root element, traverse the right subtree.

Step 6: If the element is not found in the whole tree, return NULL.

Algorithm - Deletion

In the deletion operation, we have to delete a node from the binary search tree in a way that does not violate its properties. Deletion can occur in three possible cases:

Node to be deleted is the leaf node

This is the simplest case of deleting a node in a binary search tree. Here, we will replace the leaf node with NULL and free the allocated space.

Node to be deleted has a single child node

In this case, we will replace the target node with its child and then delete that replaced child node. This means that the child node will now contain the value to be deleted. So, we will just replace the child node with NULL and free up the allocated space.

The node to be deleted has two children

This case is complex as compared to the previous two. Here we follow certain steps to delete the node:

1. First,find the inorder successor of the target node, which has to be deleted
2. Now, replace this node with the successor until the target node is at the leaf
3. Replace the target node with NULL and free up the allocated space

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Binary Search Tree and its operations such as searching and deletion has been executed and the output is verified successfully.

Ex NO: 10(a)	IMPLEMENTATION OF LINEAR SEARCH
DATE :	

Aim

To write a 'C' program to implement Linear Search.

Algorithm

Step 1: Initialize the integer variables

Step 2: Get the number of elements and its values using array and also get the search element 'x'

Step 3: Start from the leftmost element of arr[] and one by one compare x with each element of arr[]

Step 4: If x matches with an element, return the index.

Step 5: If x doesn't match with any of elements, return -1.

Step 6: Stop the program

Program

(Stick the Data Sheet)

Output

Result

Thus the 'C' program to implement the Linear search has been executed and the output is verified successfully.

Ex NO: 10(b)	IMPLEMENTATION OF BINARY SEARCH
DATE :	

Aim

To write a 'C' program to implement Binary Search.

Algorithm

Step 1: Initialize the integer variables

Step 2: Get the values from the list

Step 3: Sorted List of numbers is got as input.

Step 4: Get the target number from the user.

Step 5: Initialize first value as zero and last as n-1;

Step 6: The mid value is found.

Step 7: If the target is greater than mid then first is mid + 1 if not last is mid -1

Step 8: First is last +1

Step 9: The target is found, if its equal to a(mid) otherwise target is not found.

Program

(Stick the Data Sheet)

Output

Result

Thus the 'C' program to implement the binary search has been executed and the output is verified successfully.

Ex NO: 11	IMPLEMENTATION OF INSERTION SORT
DATE :	

Aim

To write a 'C' program to implement Insertion Sort.

Algorithm

Step 1: Start with the second element (index 1) as the key.

Step 2: Compare the key with the elements before it.

Step 3: If the key is smaller than the compared element, shift the compared element one position to the right.

Step 4: Insert the key where the shifting of elements stops.

Step 5: Repeat steps 2-4 for all elements in the unsorted part of the list

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement the Insertion sort has been executed and the output is verified successfully.

Ex NO: 12	IMPLEMENTATION OF HASH TABLE USING LINEAR PROBING
DATE :	

Aim

To write a 'C' program to implement Hash Table using Linear Probing.

Algorithm

Step 1: Start.

Step 2: Create an array of structure (i.e a hash table).

Step 3: Take a key and a value to be stored in hash table as input.

Step 4 : Corresponding to the key, an index will be generated i.e every key is stored in a particular array index.

Step 5: Using the generated index, access the data located in that array index.

Step 6: In case of absence of data, create one and insert the data item (key and value) into it and increment the size of hash table.

Step 7: In case the data exists, probe through the subsequent elements (looping back if necessary) for free space to insert new data item.

Step 8: To display all the elements of hash table, element at each index is accessed (via for loop).

Step 9: To remove a key from hash table, we will first calculate its index and delete it if key matches, else probe through elements until we find key or an empty space where not a single data has been entered (means data does not exist in the hash table).

Step 10: Stop the execution

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement Hash Table using Linear Hashing has been executed and the output is verified successfully.

Ex NO: 13	IMPLEMENTATION OF AVL TREES AND ITS ROTATIONS
DATE :	

Aim

To write a 'C' program to implement AVL Trees and its Rotations.

Balance Factor in AVL Trees

Balance factor (BF) is a fundamental attribute of every node in AVL trees that helps to monitor the tree's height.

Properties of Balance Factor

- The balance factor is known as the difference between the height of the left subtree and the right subtree.
- $\text{Balance factor}(\text{node}) = \text{height}(\text{node} \rightarrow \text{left}) - \text{height}(\text{node} \rightarrow \text{right})$
- Allowed values of BF are -1, 0, and +1.
- The value -1 indicates that the right sub-tree contains one extra, i.e., the tree is right heavy.
- The value +1 indicates that the left sub-tree contains one extra, i.e., the tree is left heavy.
- The value 0 shows that the tree includes equal nodes on each side, i.e., the tree is perfectly balanced.

AVL Rotations

To make the AVL Tree balance itself, when inserting or deleting a node from the tree, rotations are performed.

- Left – Left Rotation
- Right – Right Rotation
- Right – Left Rotation
- Left – Right Rotation

Left – Left Rotation

This rotation is performed when a new node is inserted at the left child of the left subtree.

A single right rotation is performed. This type of rotation is identified when a node has a balanced factor as +2, and its left-child has a balance factor as +1.

Right – Right Rotation

This rotation is performed when a new node is inserted at the right child of the right subtree.

A single left rotation is performed. This type of rotation is identified when a node has a balanced factor as -2, and its right-child has a balance factor as -1.

Right – Left Rotation

This rotation is performed when a new node is inserted at the right child of the left subtree.

This rotation is performed when a node has a balance factor as -2 , and its right-child has a balance factor as $+1$.

Left – Right Rotation

This rotation is performed when a new node is inserted at the left child of the right subtree.

This rotation is performed when a node has a balance factor as $+2$, and its left-child has a balance factor as -1 .

Insertion in AVL Trees

Insert operation is almost the same as in simple binary search trees. After every insertion, we balance the height of the tree.

Step 1: Insert the node in the AVL tree using the same insertion algorithm of BST.

Step 2: Once the node is added, the balance factor of each node is updated.

Step 3: Now check if any node violates the range of the balance factor if the balance factor is violated, then perform rotations using the below case.

1. If $BF(\text{node}) = +2$ and $BF(\text{node} \rightarrow \text{left-child}) = +1$, perform LL rotation.
2. If $BF(\text{node}) = -2$ and $BF(\text{node} \rightarrow \text{right-child}) = 1$, perform RR rotation.
3. If $BF(\text{node}) = -2$ and $BF(\text{node} \rightarrow \text{right-child}) = +1$, perform RL rotation.
4. If $BF(\text{node}) = +2$ and $BF(\text{node} \rightarrow \text{left-child}) = -1$, perform LR rotation.

Deletion in AVL Trees

Deletion is also very straight forward as in simple binary search trees. After deletion, restructuring the tree is done to maintain its balanced height.

Step 1: Find the element in the tree.

Step 2: Delete the node, as per the BST Deletion.

Step 3: Two cases are possible:-

Case 1: Deleting from the right subtree.

- 1A. If $BF(\text{node}) = +2$ and $BF(\text{node} \rightarrow \text{left-child}) = +1$, perform LL rotation.
- 1B. If $BF(\text{node}) = +2$ and $BF(\text{node} \rightarrow \text{left-child}) = -1$, perform LR rotation.
- 1C. If $BF(\text{node}) = +2$ and $BF(\text{node} \rightarrow \text{left-child}) = 0$, perform LL rotation.

Case 2: Deleting from left subtree.

- 2A. If $BF(\text{node}) = -2$ and $BF(\text{node} \rightarrow \text{right-child}) = -1$, perform RR rotation.
- 2B. If $BF(\text{node}) = -2$ and $BF(\text{node} \rightarrow \text{right-child}) = +1$, perform RL rotation.
- 2C. If $BF(\text{node}) = -2$ and $BF(\text{node} \rightarrow \text{right-child}) = 0$, perform RR rotation.

Program

(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement AVL Trees and its rotations has been executed and the output is verified successfully.

Ex NO: 14	IMPLEMENTATION OF B-TREES – CREATION, INSERTION AND DELETION
DATE :	

Aim

To write a 'C' program to implement B-Trees with Creation, Insertion and Deletion.

Rules for B-Tree Creation

Step 1: All leaves will be created at the same level.

Step 2: B-Tree is determined by a number of degree, which is also called "order" (specified by an external actor, like a programmer), referred to as m . The value of m depends upon the block size on the disk on which data is primarily located.

Step 3: The left subtree of the node will have lesser values than the right side of the subtree. This means that the nodes are also sorted in ascending order from left to right.

Step 4: The maximum number of child nodes, a root node as well as its child nodes can contain are calculated by this formula: $m - 1$

Step 5: Every node, except root, must contain minimum keys of $\lceil m/2 \rceil - 1$

Step 6: The maximum number of child nodes a node can have is equal to its degree, which is m

Step 7: The minimum children a node can have is half of the order, which is $m/2$

Step 8: All the keys in a node are sorted in increasing order.

Algorithm**Search Operation**

The search operation is the simplest operation on B Tree.

Step 1: Let the key (the value) to be searched by "k".

Step 2: Start searching from the root and recursively traverse down.

Step 3: If k is lesser than the root value, search left subtree, if k is greater than the root value, search the right subtree.

Step 4: If the node has the found k , simply return the node.

Step 5: If the k is not found in the node, traverse down to the child with a greater key.

Step 6: If k is not found in the tree, we return NULL.

Insert Operation

Step1: Run the search operation and find the appropriate place of insertion.

Step 2: Insert the new key at the proper location, but if the node has a maximum number of keys already:

Step 3: The node, along with a newly inserted key, will split from the middle element.

Step 4: The middle element will become the parent for the other two child nodes.

Step 5: The nodes must re-arrange keys in ascending order.

Delete Operation

The delete operation has more rules than insert and search operations.

Step 1: Run the search operation and find the target key in the nodes

Step 2: Three conditions applied based on the location of the target key, as explained in the following sections

(a) If the target key is in the leaf node

Step 2.1 Target is in the leaf node, more than min keys.

Step 2.2 Deleting this will not violate the property of B Tree

Step 2.3 Target is in leaf node, it has min key nodes

Step 2.4 Deleting this will violate the property of B Tree

Step 2.5 Target node can borrow key from immediate left node, or immediate right node (sibling)

Step 2.6 The sibling will say **yes** if it has more than minimum number of keys

Step 2.7 The key will be borrowed from the parent node, the max value will be transferred to a parent, the max value of the parent node will be transferred to the target node, and remove the target value

Step 2.8 Target is in the leaf node, but no siblings have more than min number of keys

Step 2.9 Search for key

Step 2.10 Merge with siblings and the minimum of parent nodes

Step 2.11 Total keys will be now more than min

Step 2.12 The target key will be replaced with the minimum of a parent node

(b) If the target key is in an internal node

Step 3.1: Either choose, in- order predecessor or in-order successor

Step 3.2: In case the of in-order predecessor, the maximum key from its left subtree will be selected

Step 3.3: In case of in-order successor, the minimum key from its right subtree will be selected

Step 3.4: If the target key's in-order predecessor has more than the min keys, only then it can replace the target key with the max of the in-order predecessor

Step 3.5: If the target key's in-order predecessor does not have more than min keys, look for in-order successor's minimum key.

Step 3.6: If the target key's in-order predecessor and successor both have less than min keys, then merge the predecessor and successor.

(c) If the target key is in a root node

Step 4.1: Replace with the maximum element of the in-order predecessor subtree

Step 4.2: If, after deletion, the target has less than min keys, then the target node will borrow max value from its sibling via sibling's parent.

Step 4.3: The max value of the parent will be taken by a target, but with the nodes of the max value of the sibling.

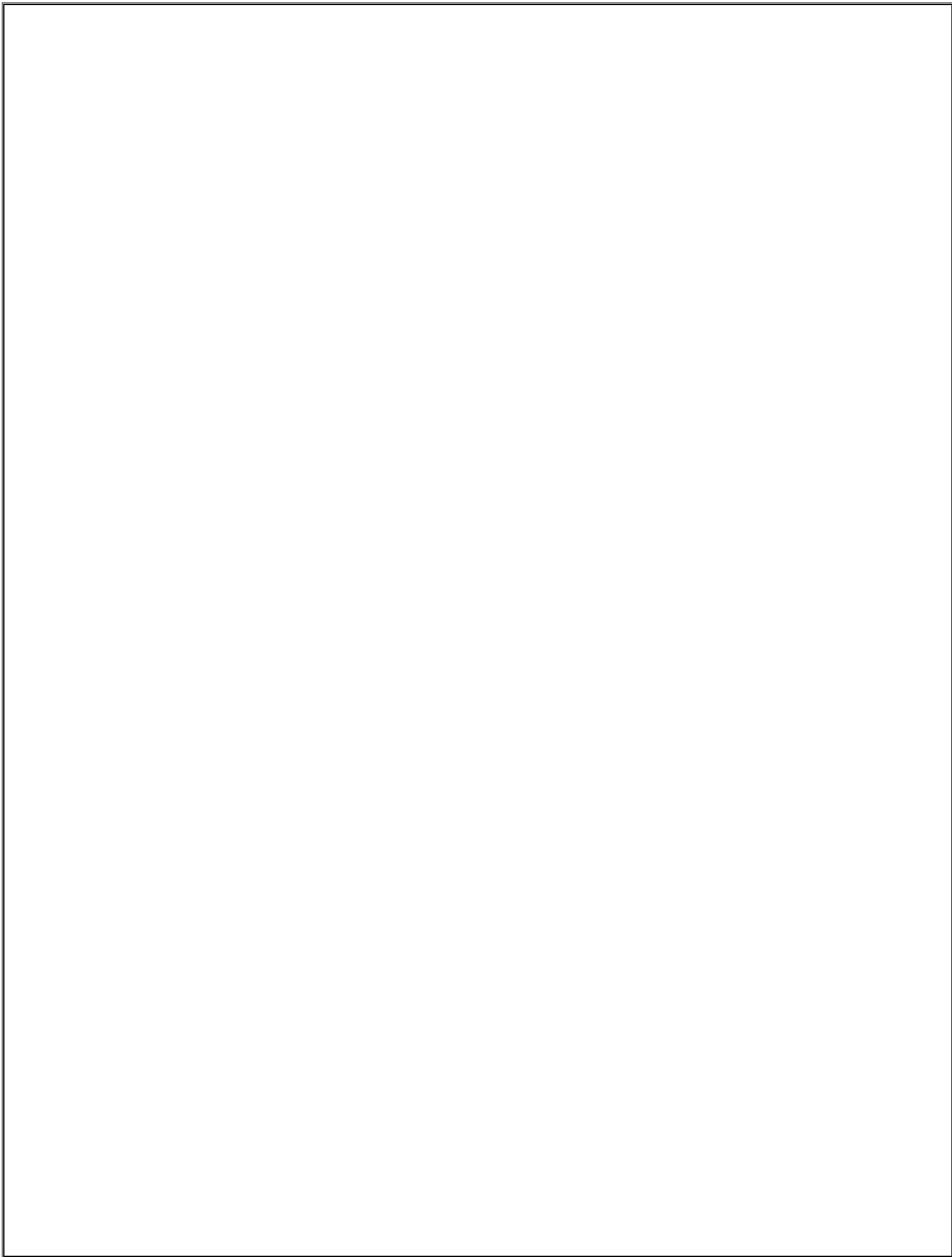
Program

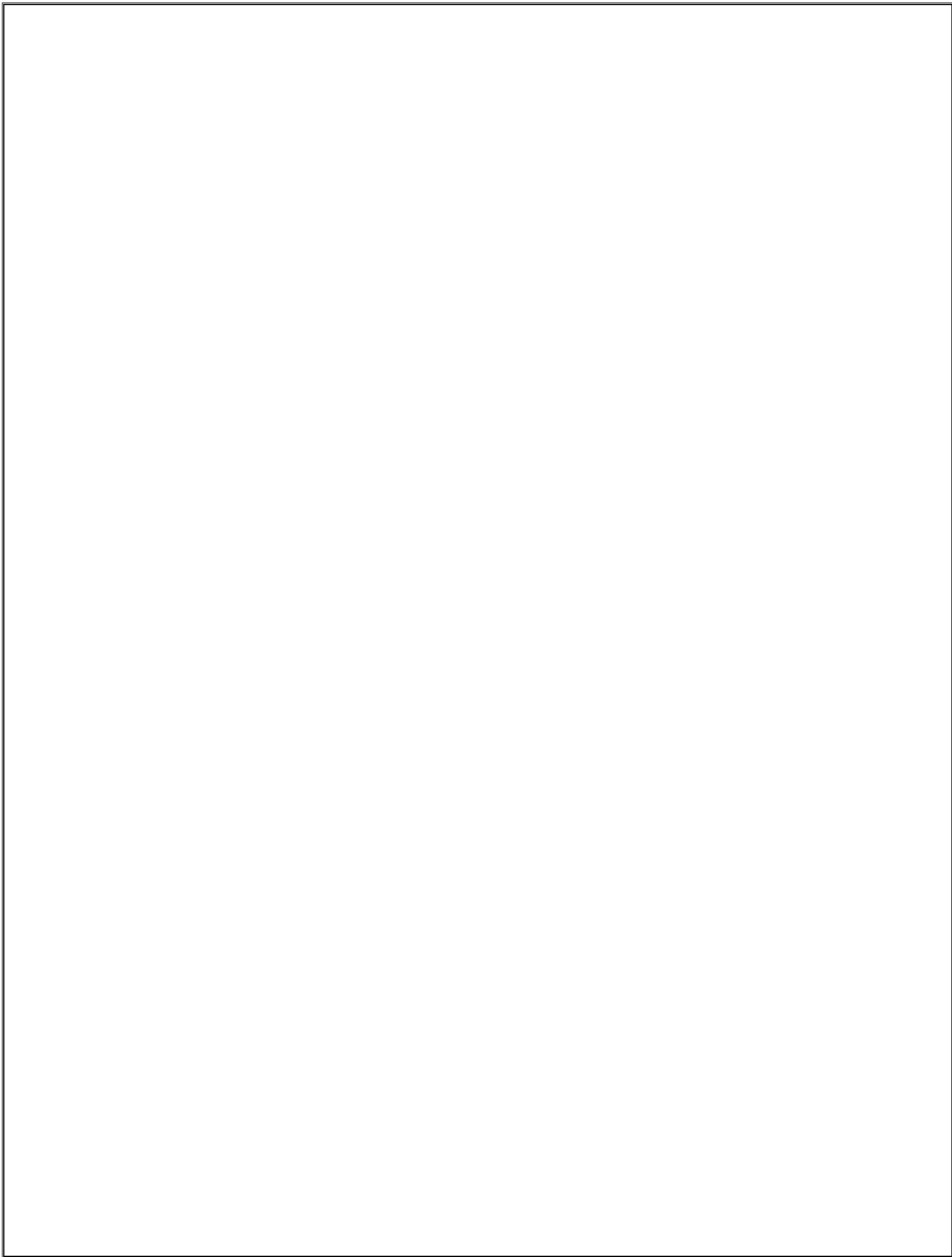
(Stick the Data Sheet)

Output

Result

Thus, the 'C' program to implement B-Trees with Creation and Traversal has been executed and the output is verified successfully.





RV University Vision and Mission

Vision

To be a world-class, tech-driven, global university for liberal education, empowering citizens of tomorrow.

Mission

- Strive for excellence in teaching, research, capacity building, community engagement and to build a reputation in the academic world as a thought leader in various disciplines; benchmark against top universities of the world.
- Use digital and other technologies relevant to teaching-learning process and to do advanced research.
- Creation of a vibrant and diverse community with faculty members and students hailing from different parts of the country and the world.
- Forge a large number of national and international collaborations to create learning opportunities and mobility for the students and faculty members.
- Make higher education accessible to all. Offer a multi-disciplinary learning environment in a diversity of knowledge streams and skills catering to the existing and emerging needs from a learner's context. To create research rewarding environment that seeks and encourages inter-disciplinary expertise and institution building enterprise among faculty members, while consistently forging cooperation between global and Indian institutions of eminence.
- Expose students to the best of theory and practice.
- Evolve a diverse talent pool of informed, self-driven, empathetic & inclusive problem solvers.



RV Vidyanikethan Post, 8th Mile, Mysuru Road, Bengaluru - 560 059
For general enquiry: enquiry@rvu.edu.in | Phone: +91 95136 73778



RV University



RVUniversity



rv-university



RV.University1



rv.university